

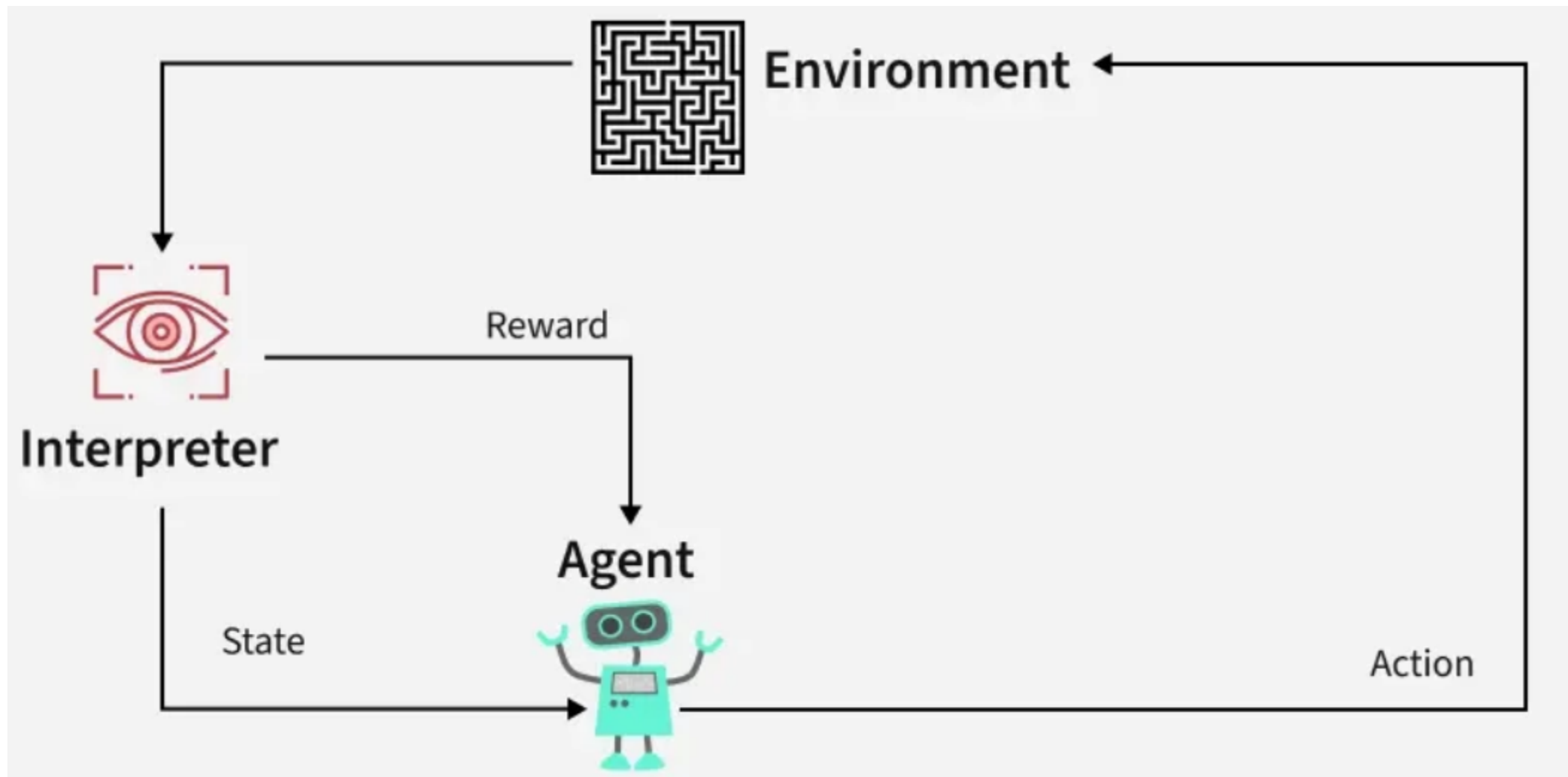
CSA4008 - APPLIED MACHINE LEARNING



MODULE IV

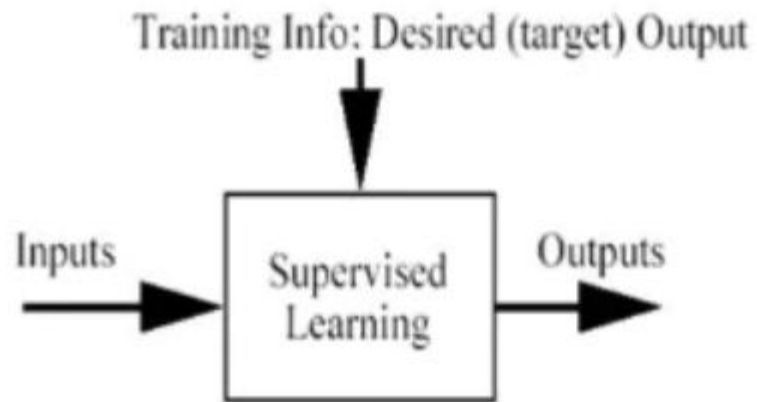
Reinforcement Learning

- **Reinforcement Learning (RL)** is a branch of machine learning that focuses on how agents can learn to make decisions **through trial and error to maximize cumulative rewards.**
- RL allows machines to learn by interacting with an environment and receiving feedback based on their actions.
- This feedback comes in the form of **rewards or penalties.**



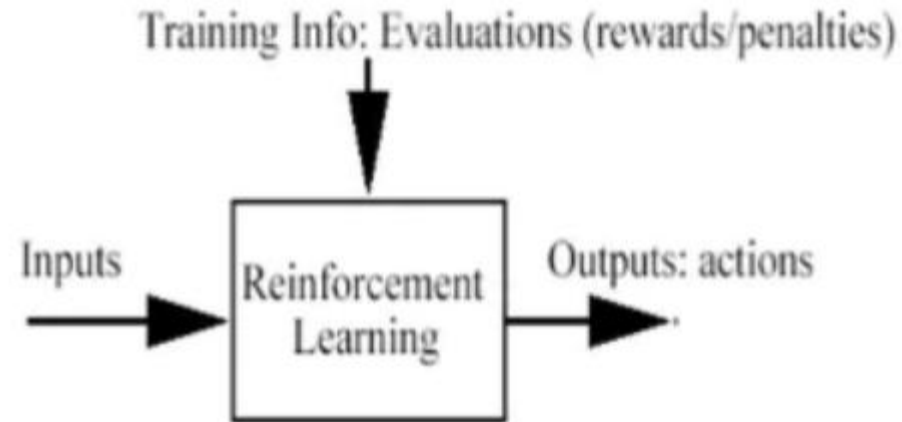
- Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal. The agent performs actions and receives feedback to optimize its decision-making over time.
- **Agent:** The decision-maker that performs actions.
- **Environment:** The world or system in which the agent operates.
- **State:** The situation or condition the agent is currently in.
- **Action:** The possible moves or decisions the agent can make.
- **Reward:** The feedback or result from the environment based on the agent's action.

Supervised Learning



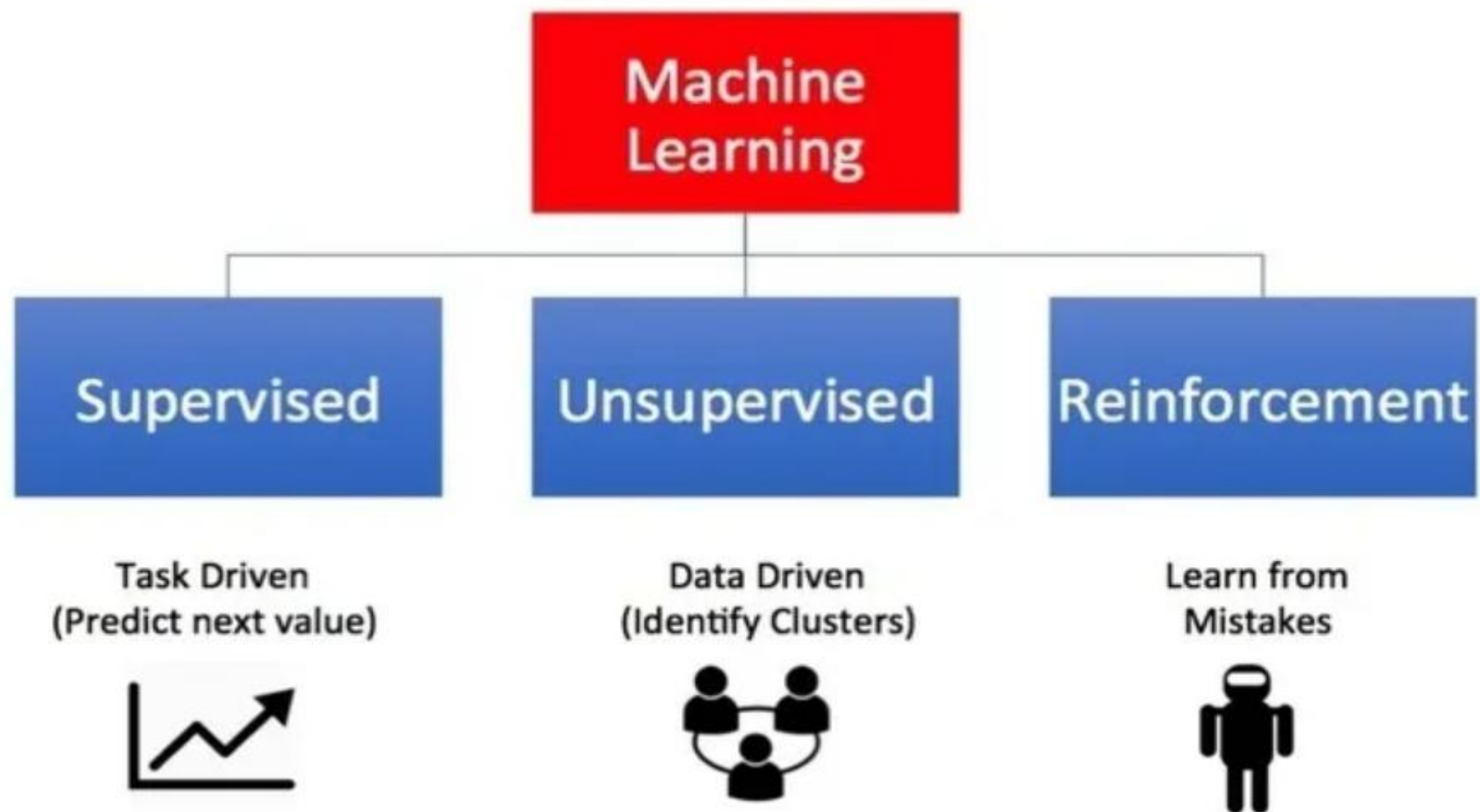
Error = (target output - actual output)

Reinforcement Learning



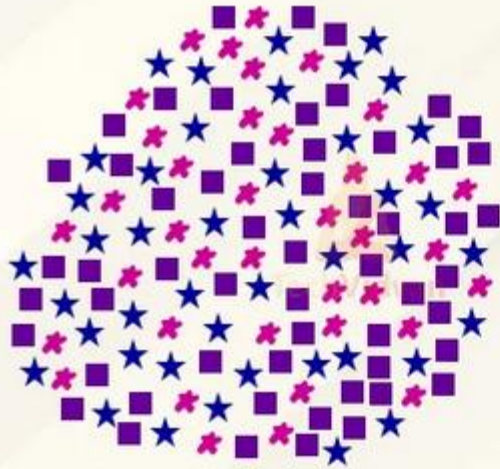
Objective: Get as much reward as possible

Types of Machine Learning



Reinforcement Learning in ML

Input Raw Data



Environment

Reward

Best Action

State

Selection of
Algorithm

Agent

Output



How Reinforcement Learning Works?

- The RL process involves an agent performing actions in an environment, receiving rewards or penalties based on those actions, and adjusting its behavior accordingly. This loop helps the agent improve its decision-making over time to maximize the **cumulative reward**.

RL components:

- **Policy:** A strategy that the agent uses to determine the *next action based on the current state*.
- **Reward Function:** A function that provides feedback on the actions taken, *guiding the agent towards its goal*.
- **Value Function:** *Estimates the future cumulative rewards* the agent will receive from a given state.
- **Model of the Environment:** A representation of the environment that *predicts future states and rewards*, aiding in planning.

Reinforcement Learning Example: Navigating a Maze

- Imagine a robot navigating a maze to reach a diamond while avoiding fire hazards. The goal is to find the optimal path with the least number of hazards while maximizing the reward:
- Each time the robot moves correctly, it receives a reward.
- If the robot takes the wrong path, it loses points.
- The robot learns by ***exploring different paths in the maze***. By trying various moves, it evaluates the rewards and penalties for each path. Over time, the ***robot determines the best route by selecting the actions that lead to the highest cumulative reward***.



- *The robot's learning process can be summarized as follows:*
- **Exploration:** The robot starts by exploring all possible paths in the maze, taking different actions at each step (e.g., move left, right, up, or down).
- **Feedback:** After each move, the robot receives feedback from the environment:
 - A positive reward for moving closer to the diamond.
 - A penalty for moving into a fire hazard.
- **Adjusting Behavior:** Based on this feedback, the robot adjusts its behavior to maximize the cumulative reward, favoring paths that avoid hazards and bring it closer to the diamond.
- **Optimal Path:** Eventually, the robot discovers the optimal path with the least number of hazards and the highest reward by selecting the right actions based on past experiences.

Types of Reinforcements in RL

1. Positive Reinforcement

- Positive Reinforcement is defined as when an event, occurs due to a particular behavior, ***increases the strength and the frequency of the behavior.***
- ***In other words, it has a positive effect on behavior.***
- **Advantages:** Maximizes performance, helps sustain change over time.
- **Disadvantages:** Overuse can lead to excess states that may reduce effectiveness.

Positive Reinforcement Example:

- If a child does homework properly every day, the parents give him chocolate.
- Here, the **chocolate** is the reward. This strengthens the child's habit of regularly completing homework.

2. Negative Reinforcement

- Negative Reinforcement is defined *as strengthening of behavior* because a negative condition is stopped or avoided.
- **Advantages:** Increases behavior frequency, ensures a minimum performance standard.
- **Disadvantages:** It may only encourage just enough action to avoid penalties.

Negative Reinforcement Example:

- If a student does not memorize lessons, the teacher gives extra work every day. But once the student starts studying well, the teacher stops giving extra work.
- Here, **removal of extra work** is the negative reinforcement. This encourages the student to continue studying properly.

Passive Reinforcement Learning

- In **Passive Reinforcement Learning**, the **agent follows a fixed policy (π)**.
- The goal is to **evaluate the given policy** – not to improve it.
- The agent **does not know**:
 - The **reward function** $R(s)$ beforehand.
 - The **transition model** $P(s'|s,a)$.
- The agent **learns the utility (value)** of each state by experience (following π).
- ***At each step, the agent only knows:***
 - Current state
 - Current reward
 - Action it must take (given by π)

- The next state is **probabilistic** (may lead to multiple states with certain probabilities).
- The **utility of a state** under policy π is:

$$U^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t(s') \right]$$

- i.e., the **expected sum of discounted rewards** until termination.
- **Goal:** Learn these utilities directly from experience without full knowledge of the environment.

- $U^\pi(s)$ – **Utility (or Value) of state s under policy π**
 - This is what we are trying to calculate or learn.
 - It represents the **expected total reward** we get if we start in state s and then follow policy π forever (or until termination).
- $\mathbb{E}[\cdot]$ – **Expectation (Average)**
 - Means we take the **average over many possible future sequences** of rewards, weighted by their probabilities (because future states can be random).
- $\sum_{t=0}^{\infty}$ – **Summation from $t = 0$ to ∞**
 - We add up rewards from the first step ($t = 0$) all the way into the future (until the episode ends).
 - If the task is episodic, this sum stops at the terminal state.

- γ^t – **Discount Factor raised to t**
 - $0 \leq \gamma \leq 1$.
 - Reduces (discounts) the importance of rewards that come later.
 - If $\gamma = 1$, all future rewards count equally;
if $\gamma < 1$, near-term rewards are valued more.
- $R_t(s')$ – **Reward received at time t (after reaching state s')**
 - Each step gives a reward signal from the environment.
 - R_0 is the immediate reward, R_1 the next one, and so on.
- s' – **Next state** (successor state)
 - Since state transitions can be random, s' is the state we reach after taking the action given by policy π .

Three common methods of estimating utility:

- 1. Direct Utility Estimation (Monte Carlo)**
- 2. Adaptive Dynamic Programming (ADP)**
- 3. Temporal-Difference (TD) Learning**

Temporal-Difference (TD) Learning

- Learns utility without explicitly building transition model.
- **Update rule:**

$$U_{\pi}(s) \leftarrow U_{\pi}(s) + \alpha [R(s) + \gamma U_{\pi}(s') - U_{\pi}(s)]$$

- $U_{\pi}(s)$ – *Utility (Value) of state s under policy π*
 - Our current estimate of how good it is to be in state s if we follow policy π .
 - This is what we are updating.
- $\leftarrow$ – *Assignment / Update operator*
 - Means "replace the old value of $U_{\pi}(s)$ with the new value on the right-hand side."
- α – *Learning rate ($0 < \alpha \leq 1$)*
 - Controls **how big the update step** is.
 - Small $\alpha \rightarrow$ slow but stable learning.
 - Large $\alpha \rightarrow$ faster learning but may overshoot.

Application of Reinforcement Learning

- **Robotics:** RL is used to automate tasks in structured environments such as manufacturing, where robots learn to optimize movements and improve efficiency.
- **Game Playing:** Advanced RL algorithms have been used to develop strategies for complex games like chess, Go, and video games, outperforming human players in many instances.
- **Industrial Control:** RL helps in real-time adjustments and optimization of industrial operations, such as refining processes in the oil and gas industry.
- **Personalized Training Systems:** RL enables the customization of instructional content based on an individual's learning patterns, improving engagement and effectiveness.

Advantages of Reinforcement Learning

- **Solving Complex Problems:** RL is capable of solving highly complex problems that cannot be addressed by conventional techniques.
- **Error Correction:** The model continuously learns from its environment and can correct errors that occur during the training process.
- **Direct Interaction with the Environment:** RL agents learn from real-time interactions with their environment, allowing adaptive learning.
- **Handling Non-Deterministic Environments:** RL is effective in environments where outcomes are uncertain or change over time, making it highly useful for real-world applications.

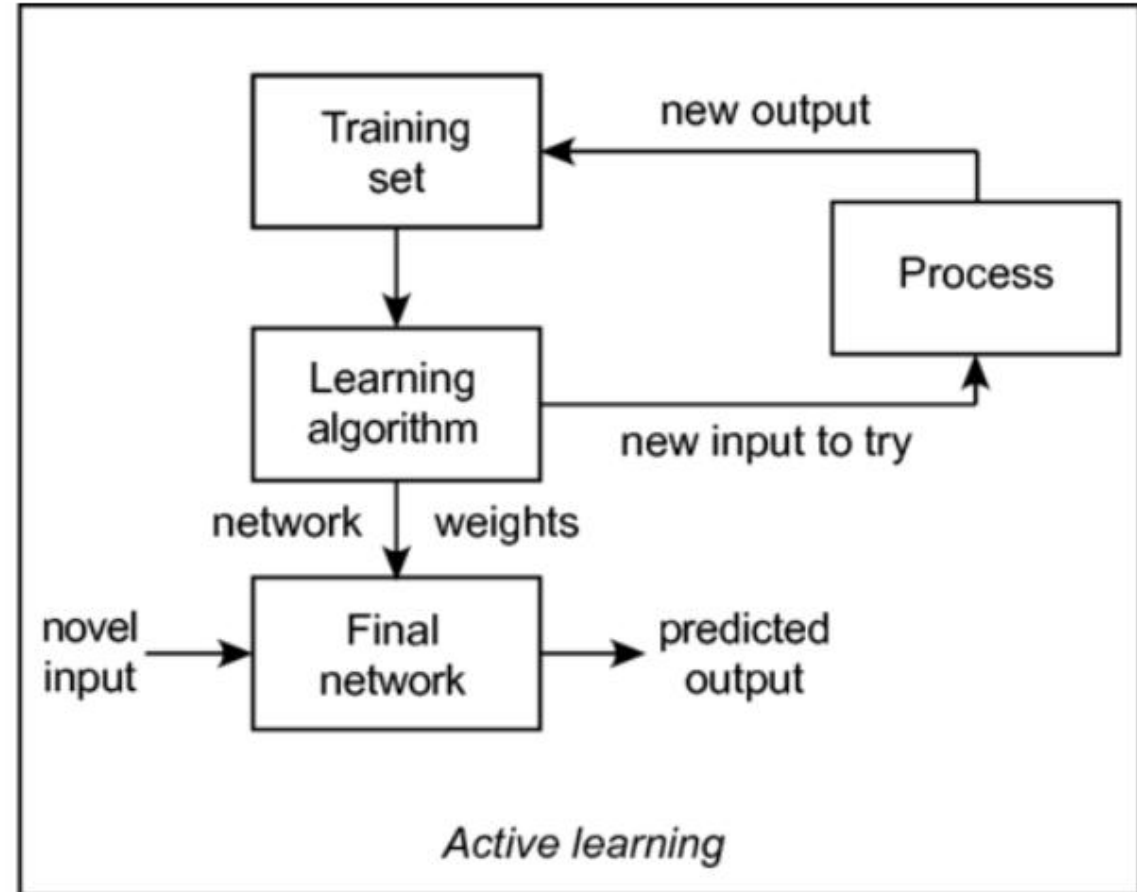
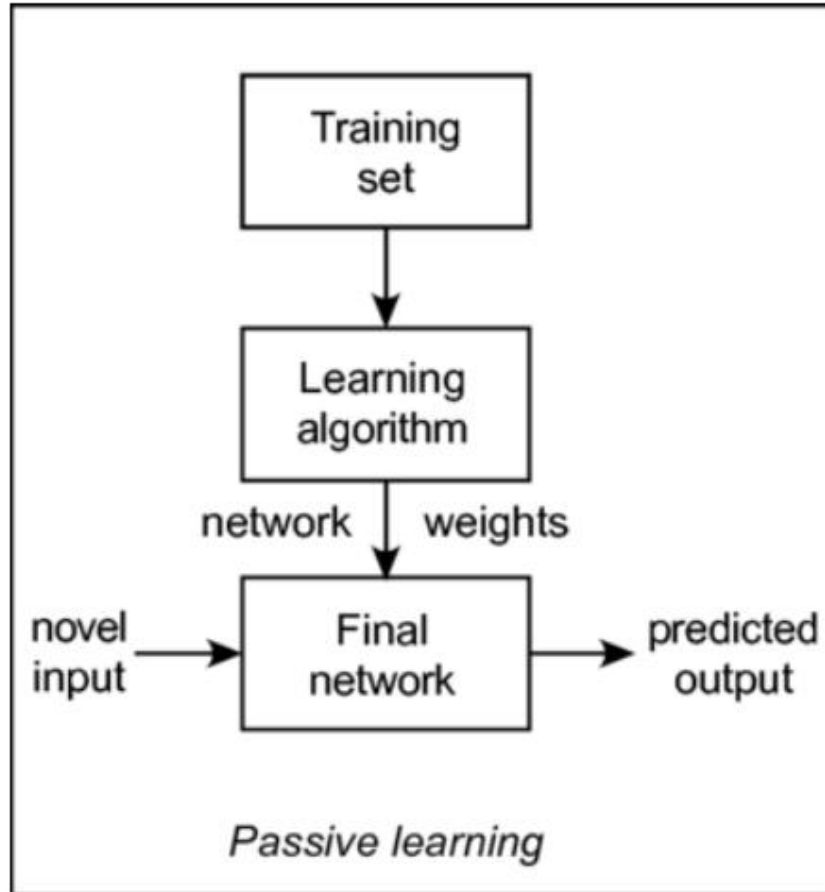
Disadvantages of Reinforcement Learning

- **Not Suitable for Simple Problems:** RL is often an overkill for straightforward tasks where simpler algorithms would be more efficient.
- **High Computational Requirements:** Training RL models requires a significant amount of data and computational power, making it resource-intensive.
- **Dependency on Reward Function:** The effectiveness of RL depends heavily on the design of the reward function. Poorly designed rewards can lead to suboptimal or undesired behaviors.
- **Difficulty in Debugging and Interpretation:** Understanding why an RL agent makes certain decisions can be challenging, making debugging and troubleshooting complex.

Passive and Active learning in Machine Learning

Passive Learning:

- Passive learning, also known as batch learning, is a method of acquiring data by processing a large set of pre-labeled data.
- In passive learning, the algorithm uses all the available data to learn and improve its performance.
- The algorithm does not interact with the user or request additional data to improve its accuracy.
- Example:- An example of passive learning is training a machine learning model to classify emails as spam or not spam. The algorithm is fed a large dataset of labeled emails and uses it to learn how to identify spam emails. Once the training is complete, the algorithm can accurately classify new emails without any further input from the user.



Active Learning:

- Active learning is a method of acquiring data where the algorithm interacts with the user to acquire additional data to improve its accuracy.
- In active learning, the algorithm starts with a *small set of labeled data and requests the user to label additional data.*
- The algorithm uses the newly labeled data to improve its performance and may continue to request additional data until a satisfactory level of accuracy is achieved.

Example:

- An example of active learning is training a machine learning model to recognize handwritten digits.
- The algorithm may start with a small set of labeled data and ask the user to label additional data that the algorithm is uncertain about.
- The algorithm uses the newly labeled data to improve its accuracy, and the process repeats until the algorithm can accurately recognize most handwritten digits.

Reading Article



Passive Learning

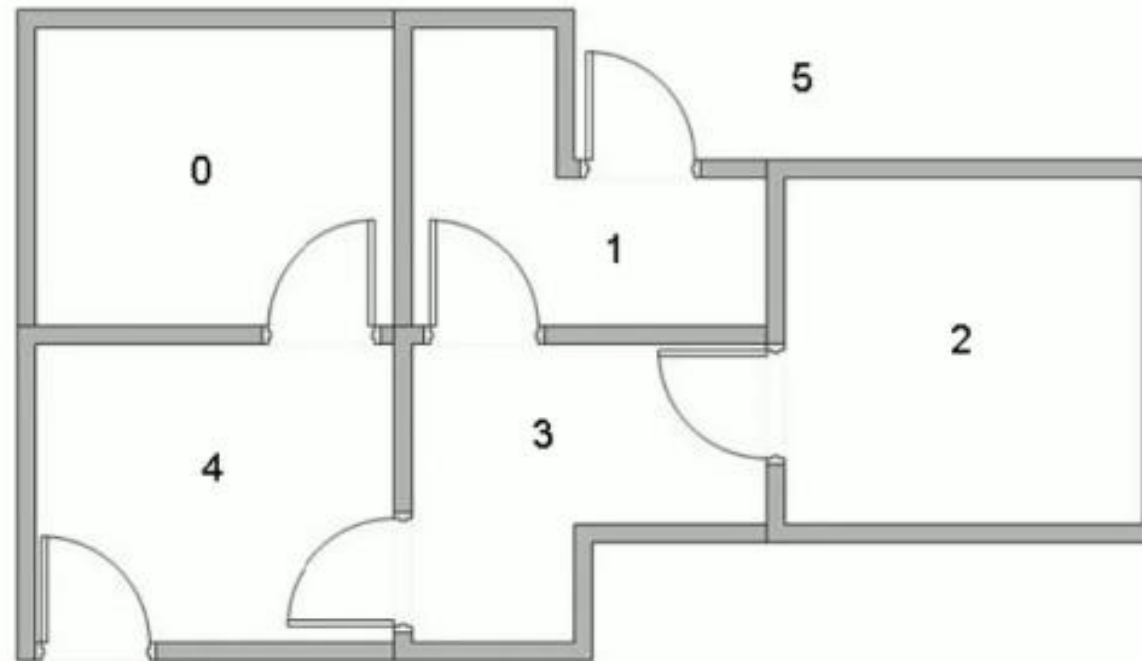
Improving Article

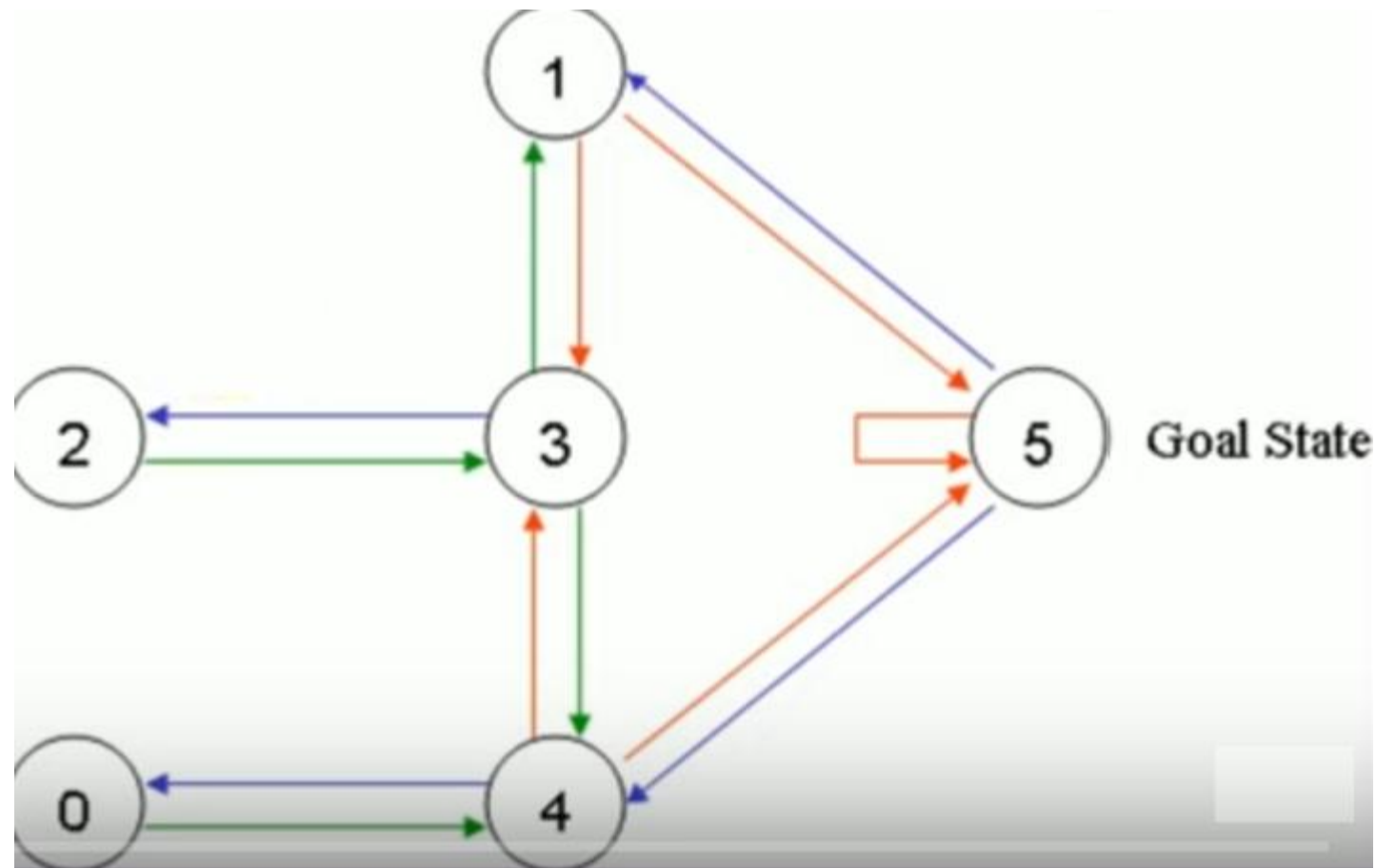
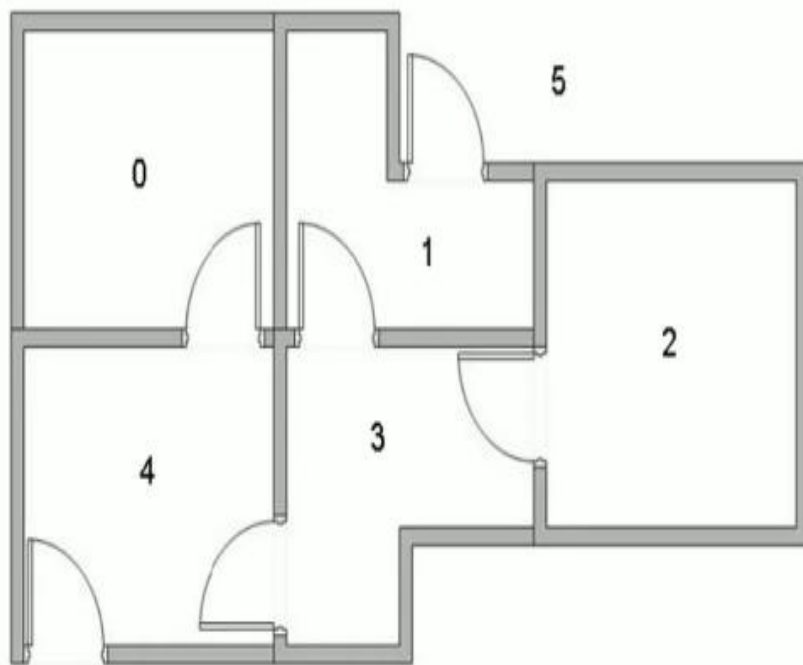


Active Learning

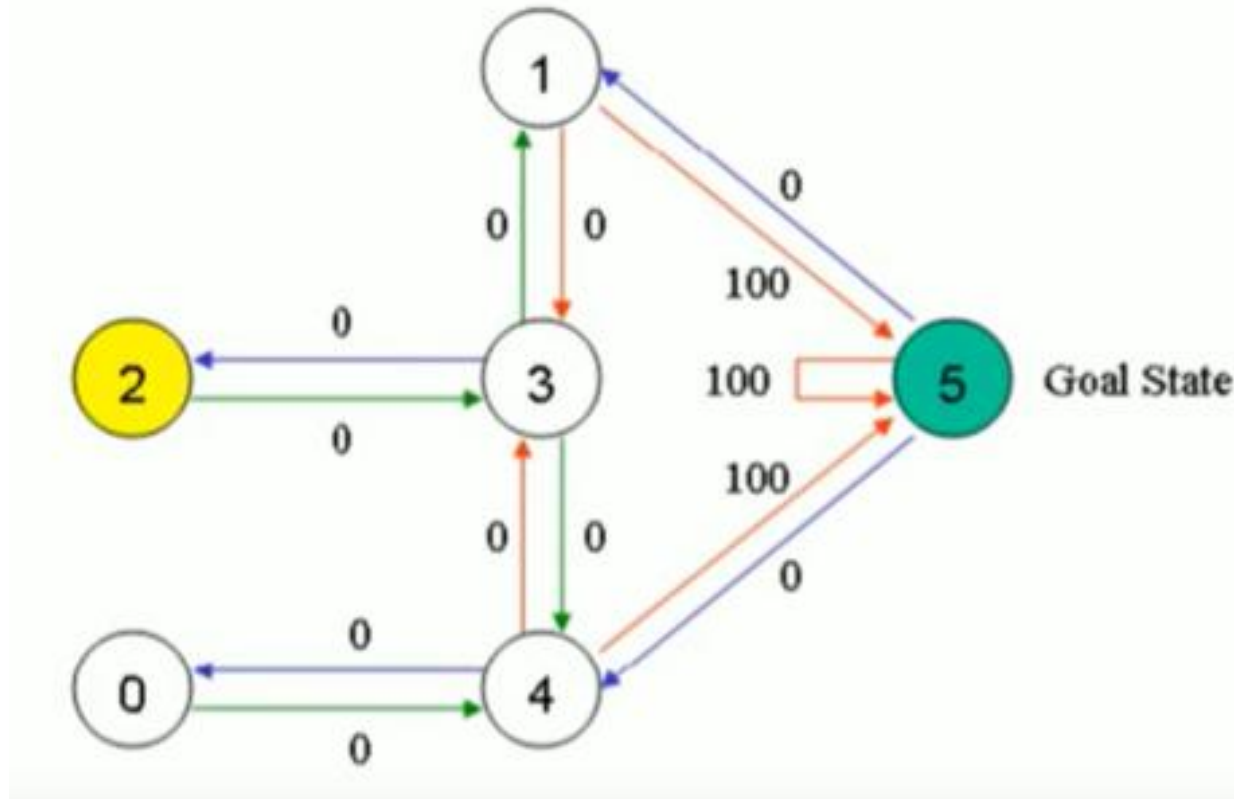
Passive Learning	Active Learning
Uses a large set of pre-labeled data to train the algorithm	Starts with a small set of labeled data and requests additional data from the user
The algorithm does not interact with the user	The algorithm interacts with the user to acquire additional data
It does not require user input after training is complete	May continue to request additional data until a satisfactory level of accuracy is achieved
Suitable for applications where a large dataset is available	Suitable for applications where labeled data is scarce or expensive to acquire

A building has **5 rooms (numbered 0 to 4)**. The outside of the building is considered as **room 5**. The rooms are connected to each other by doors as shown in the figure. Rooms **1** and **4** have doors that connect directly to the outside (room 5).

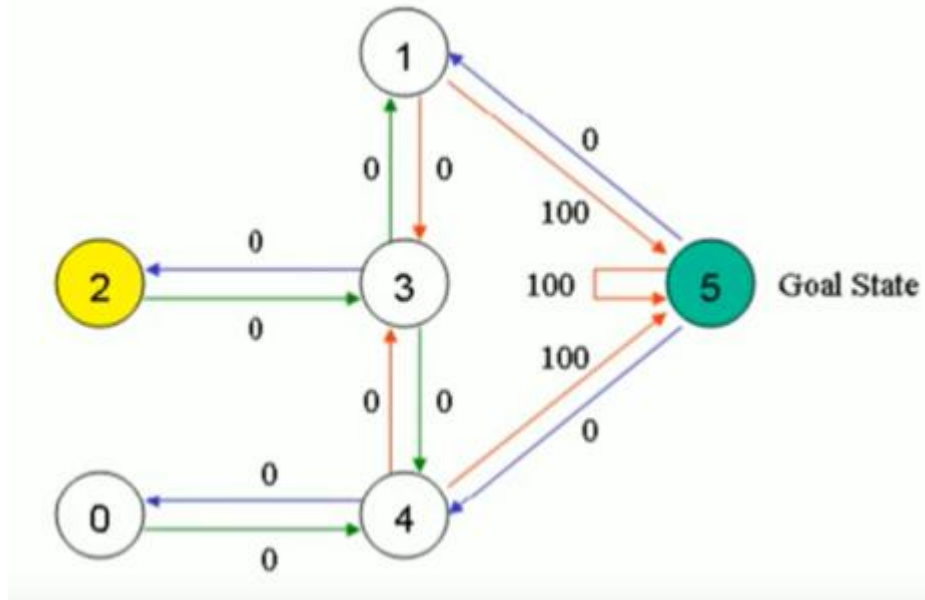




- The goal room is number 5.
- The doors that lead immediately to the goal have an instant reward of 100. Other doors not directly connected to the target room have zero reward.
- Each arrow contains an instant reward value, as shown below:



We can put the state diagram and the instant reward values into the following reward table, "matrix R". The -1's in the table represent null values (i.e.; where there isn't a link between nodes). For example, State 0 cannot go to State 1.



$$R = \begin{matrix} & \text{Action} \\ \text{State} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

Q Learning Algorithm (Quality or Action-Value" function)

- For each s, a initialize the table entry to $\hat{Q}(s, a)$ zero.
Observe the current state s .
Do forever:
 - Select an action a and execute it.
 - Receive immediate reward r .
 - Observe the new state s' .
 - Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$
- $Q(s, a)$ = Expected future reward by taking action a in state s

- **Example:**
- Suppose a robot is in **State A**.
- **Action 1** → gets a reward of +10
- **Action 2** → gets a reward of +5
- So:
- $Q(A, \text{Action1}) = 10$
- $Q(A, \text{Action2}) = 5$
- Based on this, the robot learns “**Which action is better to take?**”

- $Q(s,a)$
 - "Q-value" or **Action-Value function**
 - Represents the expected total reward when taking action a in state s .
 - It tells us “**How good is it to take this action in this state?**”
- r
 - **Immediate reward**
 - The reward received right after performing action a and moving to the new state s' .
- γ (gamma)
 - **Discount factor**
 - A value between 0 and 1: $0 \leq \gamma < 1$.
 - Determines how much importance is given to **future rewards**.
 - If $\gamma=0 \rightarrow$ The agent only considers **immediate rewards**.
 - If $\gamma=1 \rightarrow$ The agent fully considers **long-term rewards**.

- s'
 - **Next state**
 - The state reached after taking action a .
- a'
 - **Possible next actions**
 - All the actions that can be taken from the new state s' .
- $\max_{a'} Q(s', a')$
 - The **maximum Q-value** among all possible next actions a' in the new state s' .
 - It represents “**the best possible reward achievable in the future from the next state.**”
- $\leftarrow$
 - **Update operator**
 - Indicates that the old Q-value is being updated with the new calculated value.

- Learning rate = 0.8 and the initial state as Room 1.
- Initialize matrix Q as a zero matrix:

$$R = \begin{matrix} & \text{Action} \\ \text{State} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Initial state = Room 1 → chosen as a simple starting point for training, but the algorithm will eventually cover all rooms.

- **Learning Rate (α) = 0.8**

- The learning rate controls how much of the new information overrides the old Q-value when updating:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

- If α is too small (e.g., 0.1) $\rightarrow$ learning is very slow, because the agent doesn't update much from new experiences.
- If α is too large (e.g., 1.0) $\rightarrow$ the agent almost forgets past knowledge and only keeps the latest experience, which may cause instability.
- $\alpha = 0.8$ $\rightarrow$ means the agent updates its Q-values **80% from the new experience** and keeps **20% from the old knowledge**.
- It's a balanced choice to allow **fast but stable learning**.

Look at the second row (state 1) of matrix R.

There are two possible actions for the current state 1: go to state 3, or go to state 5.

By random selection, we select to go to 5 as our action.

		Action					
State		0	1	2	3	4	5
$R=$	0	-1	-1	-1	-1	0	-1
	1	-1	-1	-1	0	-1	100
	2	-1	-1	-1	0	-1	-1
	3	-1	0	0	-1	0	-1
	4	0	-1	-1	0	-1	100
	5	-1	0	-1	-1	0	100

- Now let's imagine what would happen if our agent were in state 5 (next state)
- Look at the sixth row of the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

- The next state, 5, now becomes the current state.
- Because 5 is the goal state, we've finished one episode.
- Our agent's brain now contains an updated matrix Q as:

Step 4: Use the Q-update equation

$$Q(1, 5) = R(1, 5) + \gamma \cdot \max [Q(5, 1), Q(5, 4), Q(5, 5)]$$

Currently, all Q-values = 0 (since table Q is initialized with zeros).

So,

$$Q(1, 5) = 100 + 0.8 \cdot \max(0, 0, 0)$$

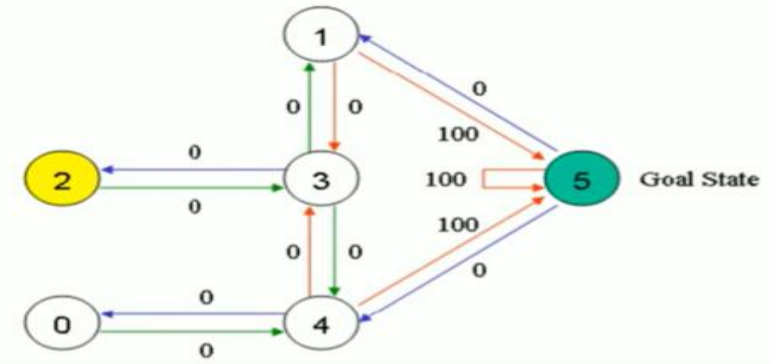
$$Q(1, 5) = 100 + 0.8 \cdot 0 = 100$$

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

- For the next episode, we randomly choose the initial state – say 3 (can go to 1, 2 & 4)

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:



- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * \text{Max}(0, 100) = 80$

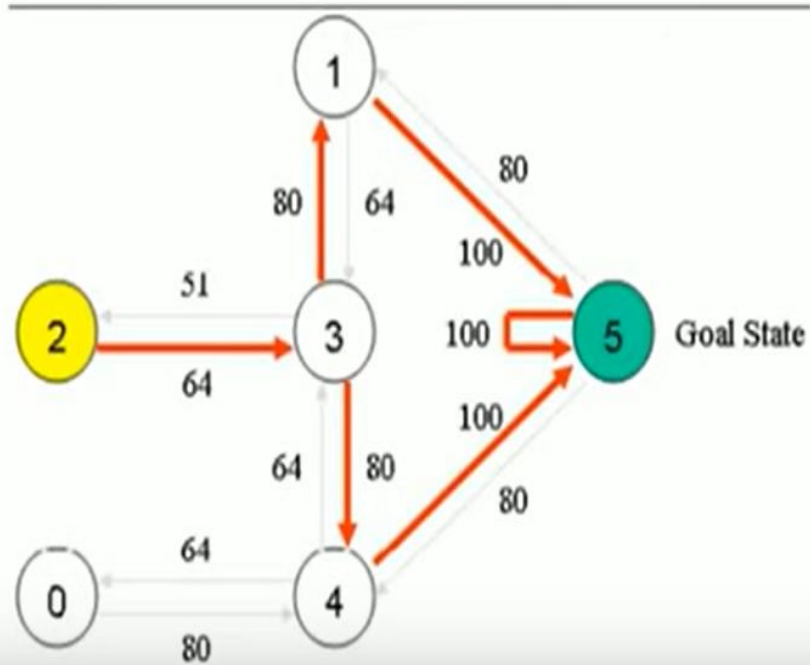
$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

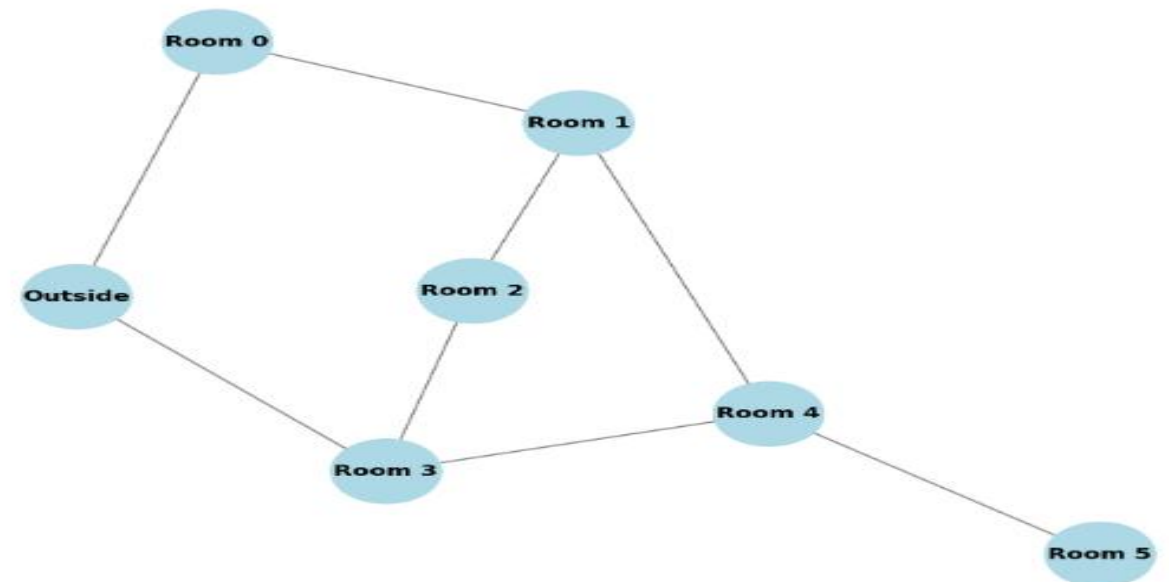
$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.



A building has **6 rooms (numbered 0 to 5)**. The outside of the building is considered as **room 6**. The rooms are connected to each other by doors as shown in the figure. Rooms **0** and **3** have doors that connect directly to the outside (room 6).

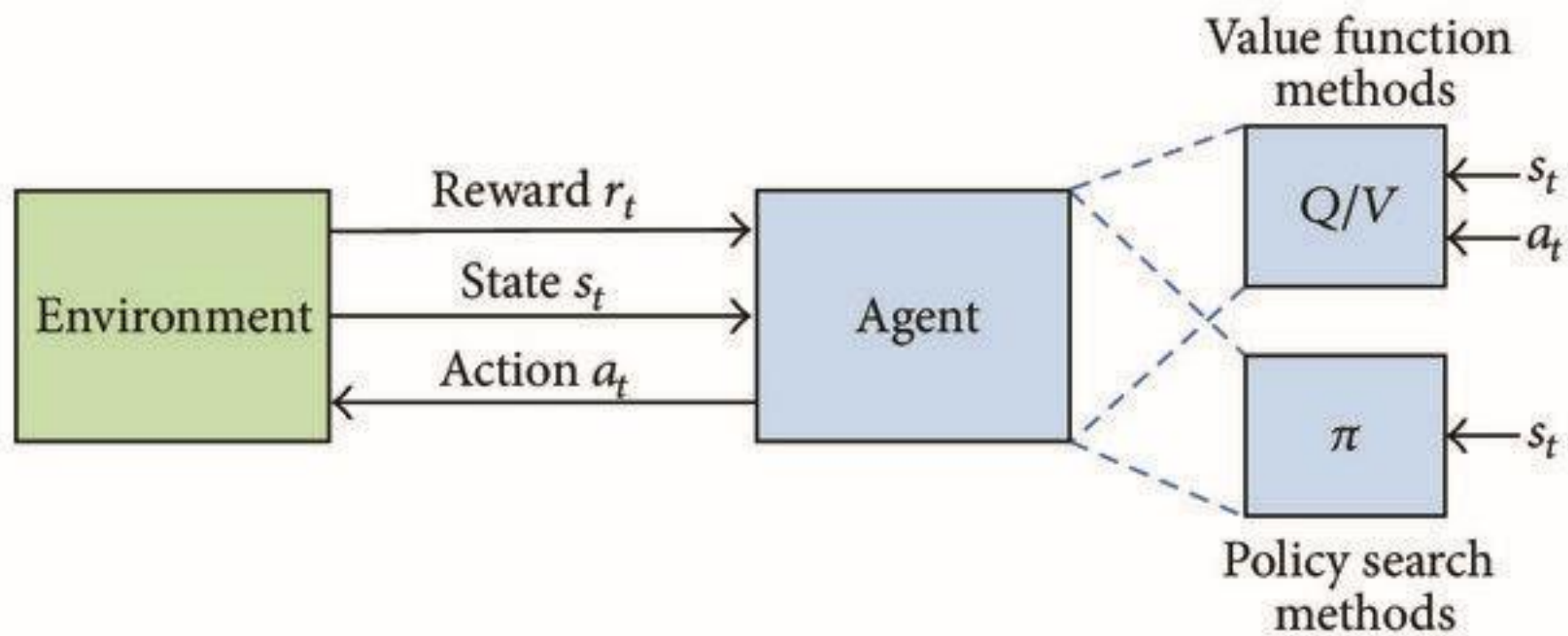
- a) Represent the building as a graph where each room is a node and doors are edges.
- b) Create the adjacency matrix for the graph.
- c) Find the shortest path from Room 5 to the outside (Room 6).
- d) Draw the final state diagram.

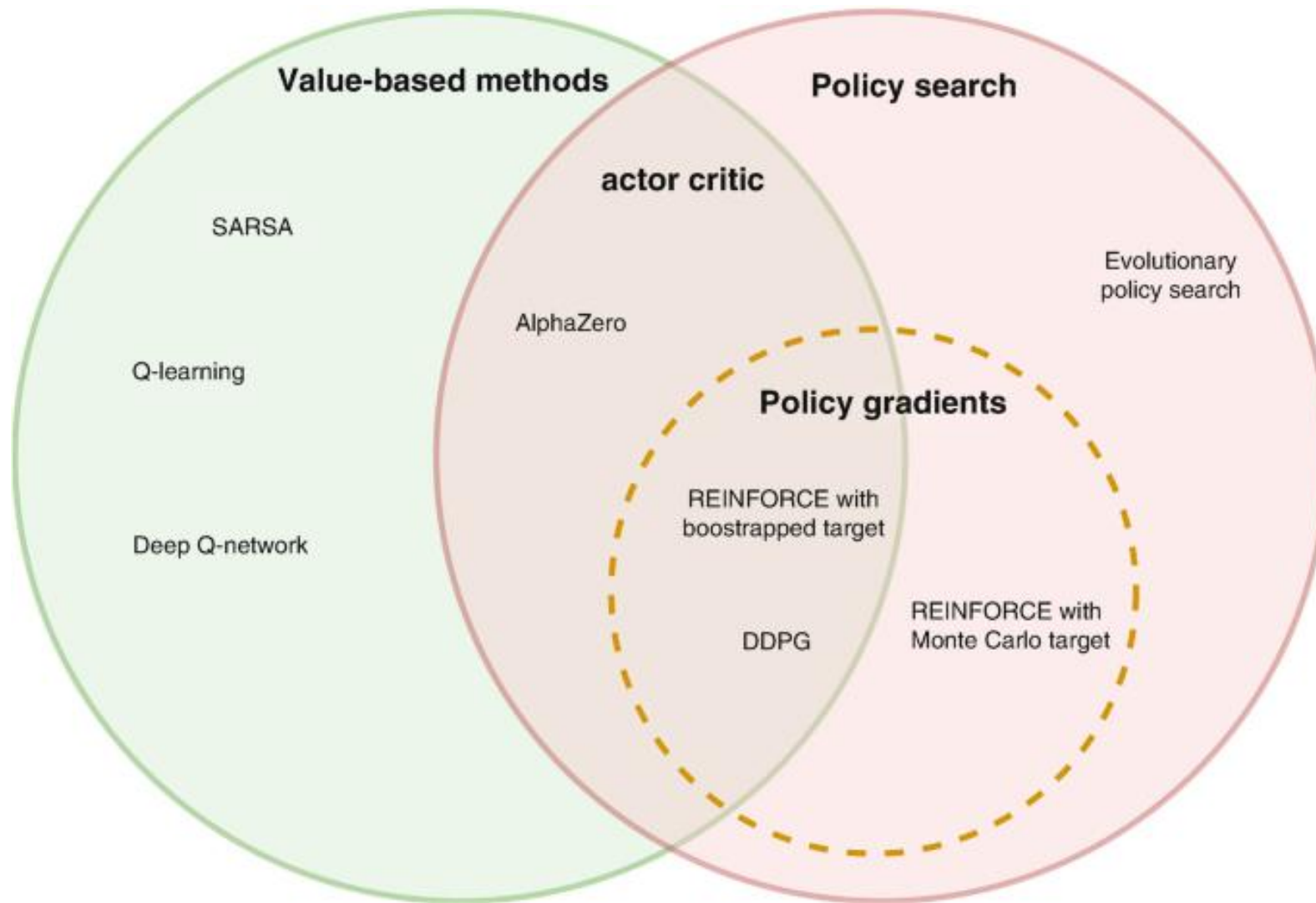


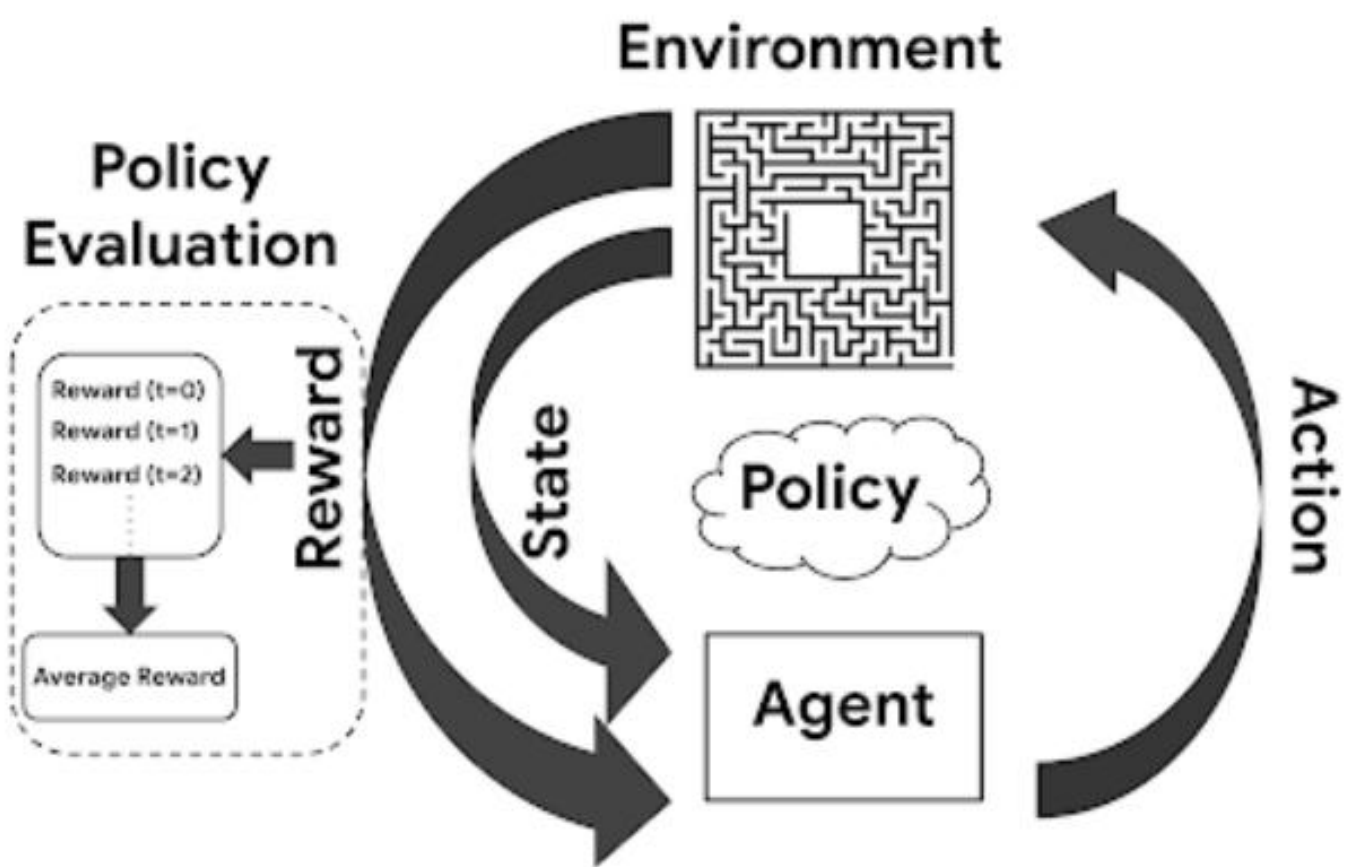
- **Note:**
- **Room 0** is connected to **Room 1** and **Room 6 (outside)**
- **Room 1** is connected to **Room 0**, **Room 2**, and **Room 4**
- **Room 2** is connected to **Room 1** and **Room 3**
- **Room 3** is connected to **Room 2**, **Room 4**, and **Room 6 (outside)**
- **Room 4** is connected to **Room 1**, **Room 3**, and **Room 5**
- **Room 5** is connected only to **Room 4**

Policy Search methods in Reinforcement Learning

- In Reinforcement Learning (RL), agents learn by interacting with their environment.
- There are **two main approaches** for learning:
 - **On-Policy Methods**
 - **Off-Policy Methods**
- Understanding the difference between these two approaches is **key to mastering RL fundamentals**.
- This topic helps build a **strong foundation** for grasping how agents learn and improve over time.







What is Policy In Reinforcement Learning (RL)?

- **Policy** = *Strategy or set of rules an agent* follows to make decisions.
- It defines a **mapping** from states → actions.
- Tells the agent **what action to take** in a given state.
- Can be:
 - **Simple** – fixed actions for each state.
 - **Complex** – uses calculations/learning to choose optimal actions.
- **Goal:** Find a policy that **maximizes cumulative reward** over time.
- **Example:** A chess player's strategy (when to attack, defend, or sacrifice pieces) is like a policy.

state → policy → action → reward

Exploration vs. Exploitation

- In Reinforcement Learning (RL), agents must balance **exploration** and **exploitation**.
- **Exploration:**
 - Trying new actions to gather information.
 - Helps discover potentially better strategies.
- **Exploitation:**
 - Choosing the best-known action based on current knowledge.
 - Aims to maximize **immediate rewards**.
- **Challenge:**
 - Too much exploration → slow learning.
 - Too much exploitation → risk of missing better strategies.
- **Goal:** Find the **optimal trade-off** for better long-term performance.

On-Policy Learning In Reinforcement Learning (RL)

- **On-Policy Methods** = Learn from what the agent is **currently doing**.
- **Example:** Teaching a robot to navigate a maze – it learns from **its own actions**.
- Similar to **learning to cook** by trying recipes yourself.
- The agent learns the **value of the current policy** (including exploration steps).
- Policy decides what action to take in every state **while learning**.
- The agent evaluates outcomes of its **present actions** and updates its strategy gradually.
- Like **hands-on practice**, it improves decision-making through **direct interaction** with the environment.

SARSA for On-Policy Learning

- **SARSA = State-Action-Reward-State-Action**
- It is a **popular on-policy method** in RL.
- The agent updates its policy based on:
 - Current state (**S**)
 - Current action (**A**)
 - Reward received (**R**)
 - Next state (**S'**) and next action (**A'**)
- **Key idea:** Learn from the actions actually taken under the current policy (including exploration).
- **No model of environment needed** – learning happens directly from experience.
- Similar to **learning on the job** – every step teaches the agent how to act next.
- **Updates the action-value function (Q-value)** using the next state **and next action** chosen by the current policy.

Mathematically, it can be represented as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where,

- $Q(s_t, a_t)$ represents the action-value function, denoting the expected cumulative future rewards of taking action a in state s .
- α is the learning rate, determining the step size for the update.
- r_{t+1} is the reward received after taking action at a_t in state s_t and transitioning to state s_{t+1} .
- γ is the discount factor, weighing the importance of future rewards.
- $Q(s_{t+1}, a_{t+1})$ is the Q-value for the next state-action pair.

The SARSA algorithm updates its Q-values based on the observed reward and the estimate of future rewards, promoting the learning of an optimal policy over successive iterations.

Off-policy learning In Reinforcement Learning (RL)

- **Off-Policy Methods** = Learn from **someone else's experience**.
- Example: A robot **watches another robot** navigate the maze and learns.
- The agent does **not need to follow** the same policy it is learning about.
- Learns the **value of the optimal policy** independent of its own actions.
- Can learn from **observations, fixed datasets, or a teaching policy**.
- Useful for **reusing past experiences** or learning from demonstrations.

Importance Sampling

- Here's a **simple point-wise version** of your text on **Importance Sampling**:
- **Importance Sampling** = A technique to estimate values from **one distribution** using data from **another distribution**.
- Commonly used in **off-policy learning** in RL.
- Helps an agent learn from **data generated by a different policy**.
- Works by giving **weights** to samples to correct for distribution mismatch.
- Allows effective learning from **external or past experiences**.
- Improves **efficiency** and **accuracy** of RL algorithms.

Q-Learning for Off-Policy Learning

- **Q-Learning** = A classic **off-policy method** in RL.
- The agent learns the **best action for each state**, even if it is not following that action while exploring.
- Example: Robot **watches other robots** succeed and learns from their moves.
- Like **reading a cookbook** – you learn recipes without actually cooking.
- Helps extract **valuable strategies from external experiences**.
- Encourages **versatile and efficient learning**.

Mathematical Update Rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Where:

- $Q(s_t, a_t)$ = Q-value for state s_t and action a_t .
- α = Learning rate (controls how much new info overrides old info).
- r_{t+1} = Reward after taking action a_t in state s_t .
- γ = Discount factor (importance of future rewards).
- $\max_a Q(s_{t+1}, a)$ = Maximum Q-value for the next state over all actions.

On-Policy vs Off-Policy in Reinforcement Learning (RL)

- **Key Difference:**
 - **On-Policy:** Learn from the **current strategy** (even if it's not optimal).
 - **Off-Policy:** Learn from a **different strategy** (can be better or more informed).
- **Exploration & Exploitation:**
 - On-policy learns while **exploring and exploiting with the same policy**.
 - Off-policy can **explore with one policy** and **learn about another policy**.
- **Analogy:**
 - On-Policy → Learning to ride a bike by actually riding, falling, and improving.
 - Off-Policy → Watching others ride bikes and learning from their experiences.

- **Examples:**

- **On-Policy:** SARSA, Temporal Difference (TD) learning.
- **Off-Policy:** Q-Learning, Deep Q-Network (DQN).

- **Advantages & Disadvantages:**

- **On-Policy:**

- ✓ Simple & intuitive (learn from your own actions).
- ✗ Less efficient (may learn from poor/unsafe actions).

- **Off-Policy:**

- ✓ More efficient (can learn from optimal actions, past data, or simulations).
- ✗ More complex (requires correcting for differences between policies).

How SARSA and Q-learning works?

- **SARSA (On-Policy):**

- Evaluates and improves the **current policy** being followed.
- Next action is chosen according to the **current policy** (including exploration).
- Updates **state-action values (Q-values)** using the actual next action taken.
- Iteratively updates values based on experiences → gradually converges to the true value of the policy.

- **Q-Learning (Off-Policy):**

- Learns about the **optimal policy** regardless of the policy being followed.
- Estimates return for the **best possible action** at the next state (greedy approach).
- Updates **state-action values (Q-values)** using the maximum value over next possible actions.
- Iterative updates converge to the optimal action-value function over time.

- **Key Difference:**

- SARSA considers the **next action chosen by the current policy**.
- Q-Learning considers the **best possible next action**, ignoring the current policy.

Off-Policy Method (Q-Learning) Example:

- **Initialization:**

- Environment is set up.
- Q-table is created with all entries initialized to **zero**.

- **Learning through episodes:**

- Agent selects actions using **epsilon-greedy strategy**.
- Takes action → observes **reward** and **new state**.

- **Q-table update (Off-Policy nature):**

- Update is based on the **maximum possible Q-value** from the next state.
- Independent of the agent's current policy or exploratory actions.
- Aims to **estimate the optimal policy**, even if actions are suboptimal.

- **Convergence:**

- Over many episodes, the Q-table **gradually converges**.
- Provides an **estimation of the optimal policy**.
- Continuous updates maximize **expected future rewards**.

Aspect	SARSA (On-Policy)	Q-Learning (Off-Policy)
Action Update	Follows the current policy	Uses the maximum Q-value at the next state
Policy Dependency	Dependent on the current policy	Independent of the current policy
Learning Approach	Learns with consideration for safety and exploration	Learns the optimal policy regardless of current actions
Example	SARSA, Temporal Difference (TD) Learning	Q-Learning, Deep Q-Network (DQN)

Problems

- A robot is in a 3-state environment: S_1, S_2, S_3
- Each state has 2 possible actions: A_1, A_2
- Learning rate $\alpha = 0.5$, discount factor $\gamma = 0.9$
- Initial Q-values: all zero

1. On-Policy (SARSA) Example

- **Step 1:** Robot is at S_1 , chooses A_1 (following its current policy)
- **Step 2:** Receives reward $R = 10$, moves to S_2 , chooses next action A_2 according to the **same policy**
- **Step 3:** Q-value update using SARSA formula:

$$Q(S_1, A_1) \leftarrow Q(S_1, A_1) + \alpha[R + \gamma Q(S_2, A_2) - Q(S_1, A_1)]$$

Calculation:

$$Q(S_1, A_1) = 0 + 0.5[10 + 0.9 * 0 - 0] = 0.5 * 10 = 5$$

- Next, the robot updates $Q(S_2, A_2)$ after taking action A_2 using the **next action chosen by its policy**.

2. Off-Policy (Q-Learning) Example

- **Step 1:** Robot is at S_1 , chooses A_1 (may follow exploration)
- **Step 2:** Receives reward $R = 10$, moves to S_2
- **Step 3:** Q-value update using Q-Learning formula (uses **max Q-value** at next state, independent of policy):

$$Q(S_1, A_1) \leftarrow Q(S_1, A_1) + \alpha[R + \gamma \max_a Q(S_2, a) - Q(S_1, A_1)]$$

Calculation:

$$Q(S_1, A_1) = 0 + 0.5[10 + 0.9 * \max(0, 0) - 0] = 0.5 * 10 = 5$$

- Next, the robot updates $Q(S_2, A)$ by considering the **best possible action** at S_2 , **ignoring the action it actually takes**.

Applications of Policy search

- **Robotics control** (robot arms, humanoids, drones)
- **Autonomous driving** (end-to-end driving policies)
- **Game playing** (continuous control games, e.g., MuJoCo, StarCraft II)
- **Personalized recommendation systems**
- **Healthcare & treatment planning**
- **Finance & portfolio optimization**
- **Industrial process control**
- **Energy management** (smart grids, HVAC systems) - Heating, Ventilation, and Air Conditioning
- **Adaptive traffic signal control**
- **Natural language generation** (dialogue systems, text-based agents)

THANK YOU!!!